

MINHAJ UNIVERSITY LAHORE



Mini Compiler — Workflow Documentation

Course: Compiler Construction (COMP338)

Group Members:

Abdul Haseeb

2023f-mulbscs-079

Roohan Ahmed

2023f-mulbscs-097

Muhammad Hamza

2023f-mulbscs-075

Zain Ali

2023f-mulbscs-084

Abdullah Ahmed

2023f-mulbscs-166

Shuaib Fakhar

2023f-mulbscs-064

MINHAJ UNIVERSITY LAHORE

1. Introduction

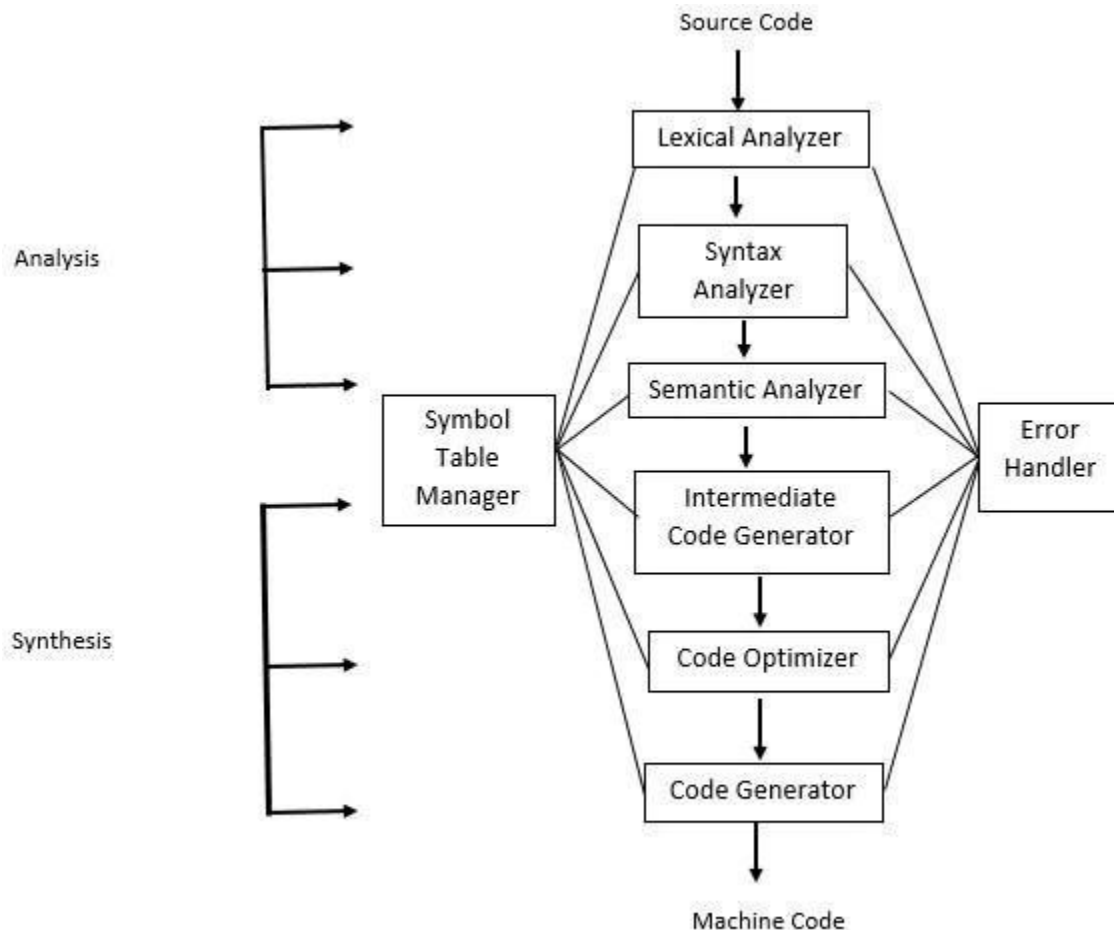
A Mini Compiler is a simplified version of a compiler that translates source code written in a high-level language into a lower-level language such as assembly, bytecode, or machine code. It implements the core phases of compilation and serves as an educational tool for understanding compiler design and language processing concepts.

2. Abstract:

This project presents the design and implementation of a Mini Compiler that demonstrates the fundamental stages of the compilation process. The compiler processes source code through lexical analysis, syntax analysis, semantic analysis, intermediate code generation, code optimization, and target code generation. Error handling is incorporated throughout each phase to identify and report issues effectively. The project provides a practical understanding of compiler construction and the transformation of source code into executable output.

3. Compiler Flow Chart

The mini compiler processes source code through the following chart:



MINHAJ UNIVERSITY LAHORE

4. Detailed Phase Descriptions

Phase 1 — Lexical Analysis (Lexer)

Input	Raw source code characters
Task	Scans the input character by character and groups them into meaningful tokens
Output	Token stream: INT, ID(x), EQ, NUM(5), PLUS, NUM(3), SEMI
Tools	Regular expressions, finite automata (DFA/NFA)
Errors	Illegal character error — e.g. using @ or # in an unsupported context
Example	int x = 5 + 3; → [INT][ID:x][=][5][+][3][;]

Phase 2 — Syntax Analysis (Parser)

Input	Token stream from the lexer
Task	Checks that the token sequence conforms to the grammar rules of the language
Output	Abstract Syntax Tree (AST) representing the program structure
Tools	Context-Free Grammar (CFG), BNF/EBNF, recursive descent or LR parsing
Errors	Syntax error — e.g. missing semicolon, unmatched parentheses
Example	AST: Assignment → Identifier(x), BinaryOp(+, 5, 3)

Phase 3 — Semantic Analysis

Input	Abstract Syntax Tree (AST)
Task	Verifies the meaning of the program: type checking, scope resolution, declarations

MINHAJ UNIVERSITY LAHORE

Output	Annotated AST with type information; symbol table populated
Tools	Symbol table (maps identifiers to type/scope), type inference rules
Errors	Type mismatch, undeclared variable, re-declaration in same scope
Symbol Table	$x \rightarrow \text{type: int, scope: local, value: uninitialized}$

Phase 4 — Intermediate Representation (IR) Generation

Input	Annotated AST
Task	Flattens the AST into a simple, linear sequence of instructions
Output	Three-Address Code (TAC) — each instruction has at most 3 operands
Format	$t1 = 5 + 3 \quad \quad x = t1$
Purpose	Platform-independent; makes optimization and code generation easier
Example	$t1 = 5 + 3 \rightarrow$ stored in temp register t1; then assigned to x

Phase 5 — Optimization (Optional)

Input	IR (three-address code)
Task	Improves efficiency of the IR without changing program behavior
Technique 1	Constant folding: evaluate constant expressions at compile time ($5+3 \rightarrow 8$)
Technique 2	Dead code elimination: remove unreachable code
Technique 3	Common subexpression elimination: avoid recomputing the same value
Output	Optimized IR: $x = 8$ (constant folded)

MINHAJ UNIVERSITY LAHORE

Phase 6 — Code Generation

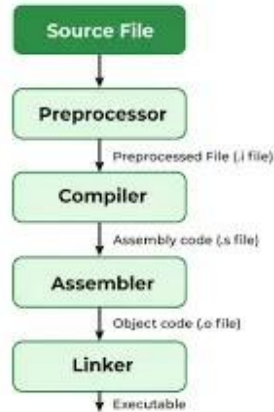
Input	Optimized IR (or plain IR if no optimization phase)
Task	Translates IR instructions into target machine instructions or assembly
Output	Assembly code, bytecode, or machine code
Register Alloc.	Maps IR temporaries (t1, t2...) to real CPU registers
Example	MOV eax, 8 MOV [x], eax
Target	Can target x86, MIPS, or a custom virtual machine / stack machine

5. Error Handling Summary

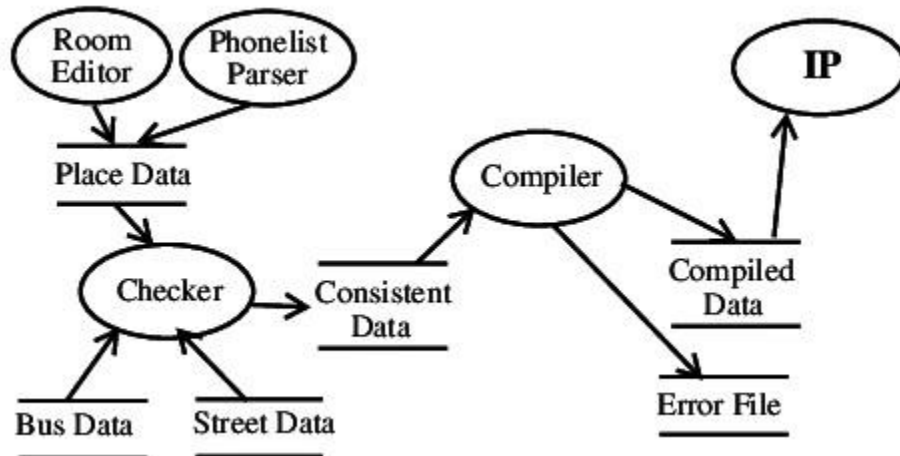
Each phase of the compiler is responsible for detecting and reporting a specific class of errors. The compiler should report as many errors as possible before stopping.

Phase	Error Type	Example
Lexer	Lexical error	Unrecognized character: @ # \$
Parser	Syntax error	Missing semicolon or mismatched brackets
Semantic	Semantic error	Type mismatch: int x = "hello"
Code Gen	Runtime error	Division by zero (if checked)

6. Mini Compiler Workflow:



7. Data Flow Diagram:



8. Conclusion

A mini compiler is an excellent project for understanding the full lifecycle of program translation. By implementing all six phases — from lexical analysis to code generation — students gain hands-on experience with formal languages, grammars, data structures (AST, symbol tables), and low-level code generation principles.

Each phase is a well-defined module with a clear input, transformation, and output, making a mini compiler both modular and testable. The techniques used in each phase — regular expressions, CFGs, type systems, three-address code — form the foundation of all modern programming language tools.